

23CSE473

NEURAL NETWORKS & DEEP LEARNING



Case study Review 2

ROAD SCENE RISK CLASSIFICATION & TRAFFIC MONITORING

TEAM NUMBER - 10				
NAME	ROLL NUMBER	GITHUB URL	COLLEGE MAIL ID	MOBILE NUMBER
Chaitanya Chitturi	CB.SC.U4CSE23214	https://github.com/iamchitturi	cb.sc.u4cse23214@cb.students.amrita.edu	9392575566
T Hema Sai	CB.SC.U4CSE23266	https://github.com/Hemasai23266	cb.sc.u4cse23266@cb.students.amrita.edu	9059587397
U Veeranjaneeyulu	CB.SC.U4CSE23351	https://github.com/Uppara-Veeranjaneeyulu	cb.sc.u4cse23351@cb.students.amrita.edu	9848267497
Charan Kola	CB.SC.U4CSE23332	https://github.com/charan-nchandr	cb.sc.u4cse23332@cb.students.amrita.edu	8977400553

SafeRoad AI — Review 2

SafeRoad AI

“Predict Risks. Prevent Accidents.”

Team 11

Chaitanya Chitturi | T Hema Sai | U Veeranjaneeyulu | Charan Kola

1. Dataset Description

Our dataset combines images from BDD100K, India Driving Dataset (IDD), and Custom YouTube Intersection Videos.

- Total Images: 6,949
- Classes: Safe (2,350), Moderate Risk (307), High Risk (4,292)
- Data Split (70-15-15 Strategy):
 - Training Set (70%): ~4,864 images for model learning.
 - Validation Set (15%): ~1,042 images for hyperparameter tuning.
 - Testing Set (15%): ~1,043 images for final performance evaluation.

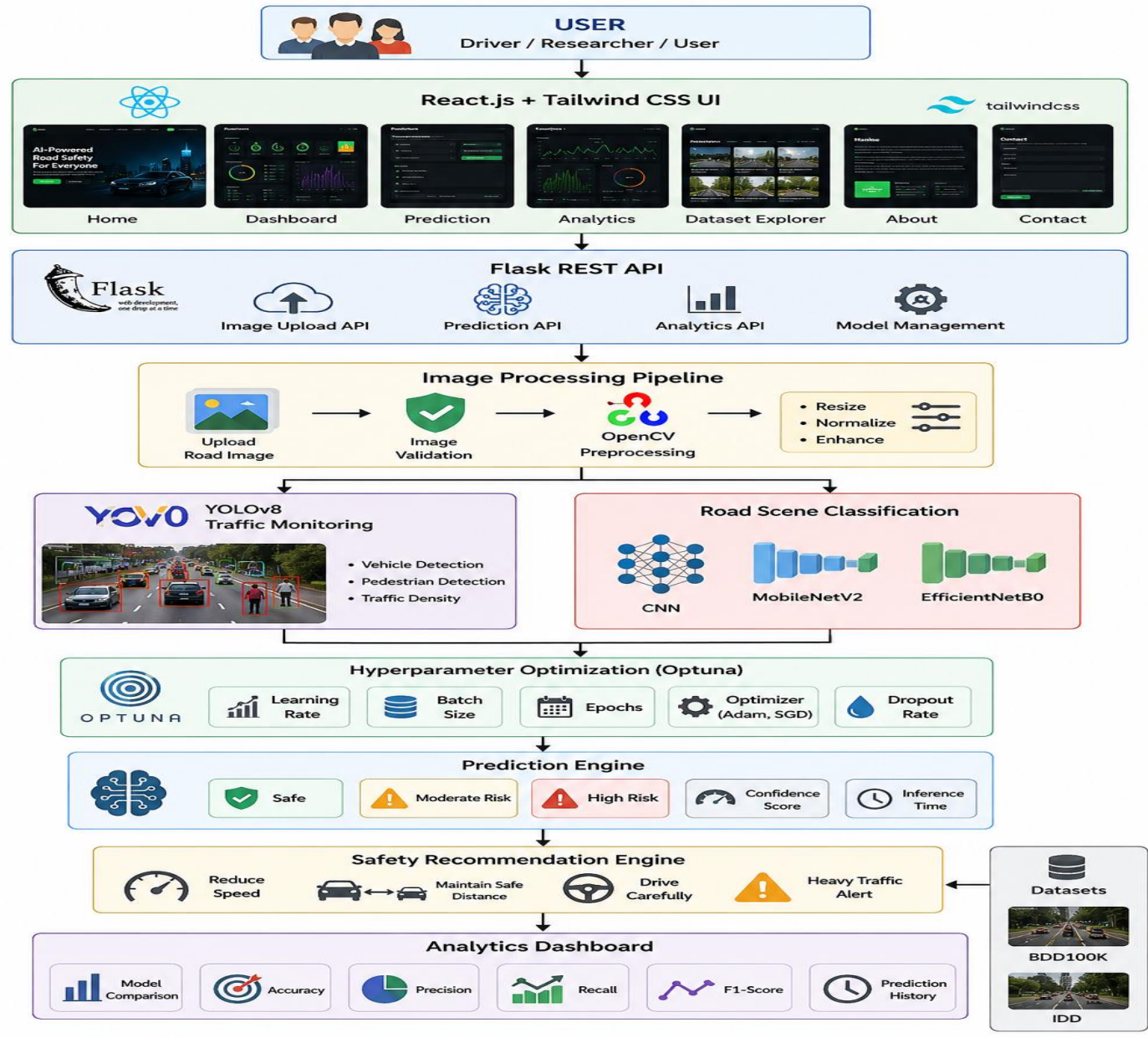
1.1 Training, Validation & Testing

The dataset is divided using a 70–15–15 strategy for training, validation, and testing while maintaining the three risk categories.

Subset	Ratio	Approx. Images	Purpose
Training	70%	4,864	Model learning / weight updates
Validation	15%	1,042	Hyperparameter tuning & model selection
Testing	15%	1,043	Final unbiased performance evaluation

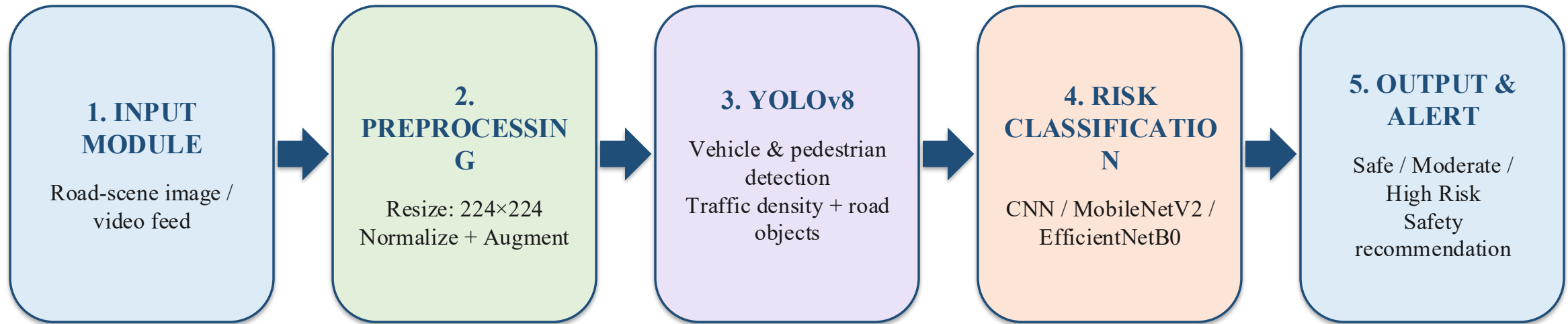
- Training Set: Used to learn model weights and visual features.
- Validation Set: Used for hyperparameter tuning and model selection.
- Testing Set: Kept unseen until final evaluation to measure generalization.

2. Overall Architecture of Product



2.1 SafeRoad AI Pipeline

From road-scene input to proactive risk prediction and web-based alerts



Pipeline Explanation

- YOLOv8 extracts traffic-related information such as vehicle count, pedestrian count, heavy vehicles, traffic signs, and direction estimation.
- CNN, MobileNetV2, and EfficientNetB0 classify the processed road scene into Safe, Moderate Risk, or High Risk.
- The final prediction and safety recommendation are displayed through the SafeRoad AI web application.

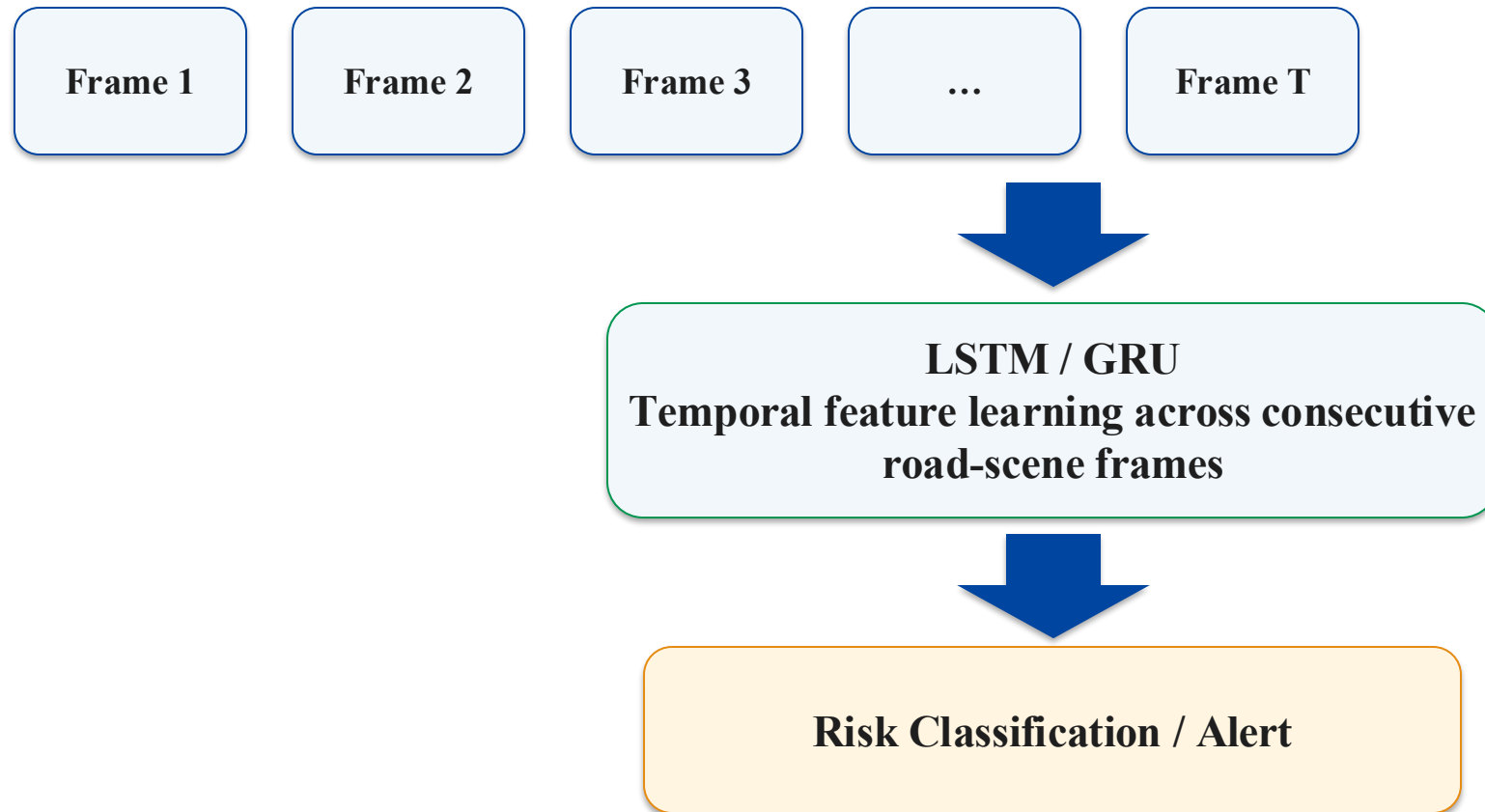
3. Deep Learning Model Architecture – CNN



Role: baseline image-classification model for Safe / Moderate Risk / High Risk.

The CNN learns spatial features directly from road-scene pixels through convolution and pooling layers, followed by classification using a three-neuron Softmax output.

3.1 RNN / LSTM Architecture – Video Extension



Important: LSTM/GRU is presented as a video/temporal extension architecture. The current SafeRoad AI training pipeline is image-based and compares CNN, MobileNetV2 and EfficientNetB0.

3.2 Models Used for Comparison

Model	Role	Key Strength
Custom CNN	Baseline image classifier	Simple, lightweight, interpretable baseline
MobileNetV2	Transfer-learning classifier	Low computational cost and fast inference
EfficientNetB0	Transfer-learning classifier	Strong accuracy–efficiency balance
YOLOv8	Traffic monitoring component	Vehicle / pedestrian detection and traffic analysis

The three classification models are evaluated on the same risk-classification task; YOLOv8 is a separate object-detection component for traffic monitoring.

4. Hyperparameter Table

Hyperparameter	Initial / Search Choice	Purpose
Optimizer	Adam	Updates model weights
Learning Rate	0.001 initial; tune with Optuna	Controls update magnitude
Batch Size	32 initial; test 16 / 32 / 64	Controls samples per training step
Epochs	30–50 with Early Stopping	Controls training duration
Loss Function	Categorical Crossentropy	Measures multi-class classification error
Dropout	0.2–0.5 search range	Reduces overfitting
Input Size	224×224	Standardized model input
LR Scheduler	ReduceLROnPlateau	Lowers LR when validation metric plateaus

5. Hyperparameter Tuning → Observations

- Learning rate: very high values can cause unstable loss; lower values generally provide smoother convergence.
- Batch size: 16 may increase training time, while 64 can increase memory use; 32 is a practical starting point.
- Epochs: use Early Stopping to stop when validation performance stops improving.
- Dropout: tune to control overfitting while preserving useful features.
- Optuna: searches combinations systematically and selects configurations based on validation performance.

These are tuning observations/working expectations; final numerical observations should be replaced with results from the actual Optuna runs.

6. Performance Metrics – Definition & Observation

Metric	Definition	Observation for SafeRoad AI
Accuracy	Correct predictions / total predictions	Overall classification performance; can be misleading with class imbalance.
Precision	$TP / (TP + FP)$	Important for reducing false High-Risk alarms.
Recall	$TP / (TP + FN)$	Critical for detecting actual High-Risk scenes and minimizing missed hazards.
F1-Score	$2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$	Balanced metric, especially useful with imbalanced classes.
Inference Time	Time required for one prediction	Lower time supports near real-time deployment.

Final observation: compare CNN, MobileNetV2 and EfficientNetB0 using the same held-out test set and report the best accuracy–efficiency trade-off.

6.1 Performance Evaluation – Results to Record

Model	Accuracy	Precision	Recall	F1-Score	Inference Time
CNN	To be measured	To be measured	To be measured	To be measured	To be measured
MobileNetV2	To be measured	To be measured	To be measured	To be measured	To be measured
EfficientNetB0	To be measured	To be measured	To be measured	To be measured	To be measured

- Generate confusion matrix and classification report for each model.
- Plot training/validation accuracy and loss curves.
- Use Optuna study results to report the best hyperparameter configuration.
- Deploy the selected best model through the existing React frontend and Flask API.

7. Current Progress & Next Steps

- Completed: React + Tailwind UI with Home, Dashboard, Prediction, Analytics, Dataset, About and Contact pages.
- Completed: BDD100K + IDD dataset collection/metadata organization.
- In progress: preprocessing, stratified split and deep-learning model training.
- Next: train CNN, MobileNetV2 and EfficientNetB0; run Optuna tuning; evaluate on the held-out test set.
- Final integration: connect trained models to Flask API and replace simulated prediction endpoints in the web application.

Live frontend: <https://saferoad-ai-one.vercel.app/>

Thank You

SafeRoad AI — Predict Risks. Prevent Accidents.